

Requirement Analysis Phase

Customer Journey Map

Date	October 03,2026
Team ID	SWTID-2026-7210
Project Name	Script Controlled ACL - Restrict Record Access Based On Field Value
Maximum Mark	4 Marks

Customer Journey Map Matrix Report

A Customer Journey Map provides a deep structured visual framework capturing step-by-step end-user experiences, operational touchpoints, internal friction, and key areas of opportunities. For this framework, the mapping analyzes the end-to-end journey of Security Administrators, Compliance Officers, and Application Developers trying to implement dynamic Data Access Constraints using a Script-Controlled Access Control List (ACL) engine based on contextual field values.

Dimension Metric	Stage 1: Entice (Awareness)	Stage 2: Enter (Initialization)	Stage 3: Browse & Design	Stage 4: Engage (Execution)	Stage 5: Exit & Extend
Steps & Process Experience	Compliance Auditor reviews legacy systemic data leakage reports. Discovers static role-based parameters are failing to keep sensitive fields safe.	Security Architect enters the central policy setup console environment. Prepares a custom sandbox container path to initialize script routing rules safely.	Developer outlines logic maps connecting resource attributes to security parameters. Drafts expression hooks checking conditional flags like Status == 'Confidential'.	Engine triggers background runtime queries. Dynamically parses criteria, cross-references transaction properties, and blocks or masks row visibility instantly.	The system exports signed cryptographic execution logs to audit storage. Demonstrates continuous programmatic compliance during external governance reviews.
Interactions & Touchpoints	Internal vulnerability summary dashboards, data schema layout sheets, and corporate regulatory requirement checklists.	Cloud identity access consoles, administrative policy manager portals, and security dashboard environments.	Code workbench scripts, visual condition builders, enterprise repository environments, and test validation datasets.	Database evaluation triggers, API query gateways, and dynamic context interception filter services.	Cryptographic compliance file servers, historical log analyzers, and regulatory tracking reporting panels.
Primary Goals & Motivations	Help me identify precise data leak risks without	Help me initialize sandboxed runtime logic	Help me write clean conditional structures	Help me process instant record restriction	Help me archive persistent operational

	completely disrupting regular database team productivity patterns.	paths safely without risking global production engine crashes or downtime.	without hardcoding static roles inside complex database objects.	queries while keeping server execution latency spikes at near-zero rates.	trails to seamlessly satisfy corporate validation check parameters.
Positive Moments	Clear visualization of baseline data leaks highlights clear reasons to migrate toward advanced script logic rules.	Successful integration of policy isolation controls prevents accidental global configurations from impacting active staff.	Visual criteria validation blocks allow developers to run localized syntax evaluations before public execution steps.	Automated record filtration accurately isolates restricted columns based entirely on changing field updates.	Comprehensive log outputs provide full audit tracking without requiring manual administrative data aggregation.
Negative Moments / Friction	Legacy architectures offer no dynamic field filters, causing massive anxiety regarding data exposure risks.	Initial configuration interface options present high complexity barriers, leading to initial administration confusion.	Syntax error messages lack explicit root-cause descriptions, increasing standard logic debugging intervals.	Complex script executions produce processing lags, raising serious performance degradation concerns.	Heavy trail records trigger large storage consumption trends, threatening overall operational capacity ceilings.
Areas of Opportunity	Implement automated scanning matrices to flag static configuration weaknesses before they manifest into breaches.	Introduce setup validation wizards to guide administrators through isolated configuration paths seamlessly.	Incorporate an inline code completion assistant to immediately point out logical structural code bugs.	Build a fast localized data cache layer to handle recurring attribute checks without stressing database systems.	Deploy optimized log-compression procedures to retain extensive audit histories without straining standard storage allocations.